CSEE 4121 — Practice Midterm 2

Computer Systems for Data Science | Spring 2026 | Harder Difficulty

- **Time:** 90 minutes
- **Questions:** 20 — mix of MCQ, short answer, SQL, and trace problems
- **Total Points:** 100
- **Difficulty:** Slightly above midterm level

1 PERFORMANCE 5 pts

A program spends 80% of its time in a parallelizable section. You have unlimited cores. What is the maximum possible speedup according to Amdahl's Law?

- (●) a) 5x
- ( ) b) 10x
- ( ) c) 80x
- ( ) d) Infinity

Correct! Amdahl's Law with infinite speedup on the parallel portion: Speedup = 1 / (1 - 0.8) = 1 / 0.2 = 5x. The 20% sequential portion is the bottleneck — no matter how many cores you throw at the parallel part, you can never exceed 5x. This is the key insight of Amdahl's Law: the sequential fraction sets a hard ceiling.

2 WAL 5 pts

A system writes data pages to disk before writing the corresponding log records. After a crash, which ACID property is violated and why?

- ( ) a) Durability — committed transactions can't be replayed
- (●) b) Atomicity — modified data is on disk but there's no log to undo incomplete transactions
- ( ) c) Isolation — other transactions can see partial writes
- ( ) d) Consistency — constraints are not checked before writing

Correct! WAL Rule: log must be written BEFORE data pages. If data is written first and the system crashes before the log is written, modified data is on disk but there's no log record to undo it. The database is stuck with partial modifications from an incomplete transaction — Atomicity is violated.

3 ISOLATION 5 pts

PostgreSQL's default isolation level is Read Committed. Which of the following anomalies can still occur at this level? Select the most complete answer.

- (●) a) Dirty reads only
- ( ) b) Lost updates, unrepeatable reads, and phantom reads
- ( ) c) Phantom reads only
- ( ) d) None — Read Committed prevents all anomalies

Incorrect (you picked: A, correct: B). Read Committed only prevents dirty reads (reading uncommitted data). It does NOT prevent lost updates (two concurrent writes overwriting each other), unrepeatable reads (re-reading a row gets a different value), or phantom reads (re-running a query gets different rows). These are all still possible at this isolation level, which is PostgreSQL's default.

4 SQL / SUBQUERIES 5 pts

```
Product(PName, Price, Category, Manufacturer) Company(CName, Country, NoOfEmployees)
```

Write a SQL query to find all products that are **more expensive than every other product in the same category** (i.e., the most expensive product per category). Do NOT use GROUP BY.

```
Write your SQL here...
```

No answer provided.

Model answer:

```
-- Using NOT EXISTS:
SELECT p1.PName, p1.Price, p1.Category
FROM Product p1
WHERE NOT EXISTS (
    SELECT * FROM Product p2
    WHERE p2.Category = p1.Category
    AND p2.Price > p1.Price
);

-- OR using ALL:
SELECT PName, Price, Category
FROM Product p1
WHERE Price >= ALL (
    SELECT Price FROM Product p2
    WHERE p2.Category = p1.Category
);
```

This requires a correlated subquery. For each product, the inner query checks if any other product in the same category has a higher price. NOT EXISTS or >= ALL both work. You cannot use GROUP BY per the instructions — this tests correlated subquery mastery.

5 SQL / AGGREGATION 5 pts

A table `employees(id, name, dept)` has 100 rows. The `dept` column is NULL for 15 employees. What does `SELECT COUNT(*), COUNT(dept) FROM employees` return?

- ○ a) 100, 100
- ◉ b) 100, 85
- ○ c) 85, 85
- ○ d) 85, 100

Correct! COUNT(*) counts all rows regardless of NULL values — returns 100. COUNT(dept) counts only non-NULL values in the dept column — returns 100 - 15 = 85. This is a critical distinction: COUNT(*) counts rows, COUNT(column) counts non-NULL values in that column.

6 SQL / JOINS 5 pts

Table `A` has 10 rows. Table `B` has 8 rows. No rows in A match any row in B on the join condition. How many rows does `A FULL OUTER JOIN B` return?

- ○ a) 0
- ○ b) 10
- ◉ c) 18
- ○ d) 80

Correct! FULL OUTER JOIN guarantees all rows from both tables appear. With zero matches, every row from A appears with NULLs for B's columns (10 rows) and every row from B appears with NULLs for A's columns (8 rows). Total = 10 + 8 = 18. An INNER JOIN would return 0 rows. A LEFT JOIN would return 10. A RIGHT JOIN would return 8.

7 RELATIONAL MODEL 5 pts

A `Students` table has a primary key `id`. An `Enrollments` table has a foreign key `student_id` referencing it with `ON DELETE CASCADE`. If you delete a student, what happens to their enrollments?

- ○ a) The delete is rejected because enrollments reference the student
- ○ b) The enrollments' student_id is set to NULL

○ c) All enrollments for that student are automatically deleted

○ d) The enrollments remain unchanged with a dangling reference

Incorrect (you picked: none, correct: C). ON DELETE CASCADE means when a parent row is deleted, all child rows referencing it are automatically deleted too. This is dangerous — deleting one student silently wipes their entire enrollment history. ON DELETE RESTRICT would reject the delete. ON DELETE SET NULL would set student_id to NULL in the enrollments.

8 PROTOCOL BUFFERS 5 pts

Why should you avoid using `required` fields in Protocol Buffers and prefer `optional`?

○ a) Required fields use more bytes on the wire

◉ b) Required fields break backward compatibility — old clients crash if a new required field is added

○ c) Required fields are slower to serialize

○ d) Required fields cannot be numeric types

Correct! If you add a new required field, old clients that don't know about this field will fail to serialize valid messages (they can't provide the required field). This breaks backward compatibility. Optional fields are safe — old clients simply don't include them, and new clients handle their absence gracefully.

9 LOCKING 5 pts

Under Strict 2PL, when are locks released?

○ a) Immediately after the operation that needed the lock completes

○ b) At the end of the shrinking phase, before commit

◉ c) All at once when the transaction commits or aborts

○ d) In reverse order of acquisition

Correct! Strict 2PL is stricter than basic 2PL. In basic 2PL, locks can be released before commit (shrinking phase). In STRICT 2PL, ALL locks are held until the transaction commits or aborts, then released all at once. This prevents cascading aborts (where aborting one transaction forces aborting others that read its uncommitted data).

10 DEADLOCKS 5 pts

Three transactions under Strict 2PL:

```
T1: holds X(A), requests S(B)
T2: holds X(B), requests S(C)
T3: holds X(C), requests S(A)
```

Draw the waits-for graph. Is this a deadlock? If so, what is the cycle?

```
Describe the waits-for graph and whether there is a deadlock...
```

No answer provided.

Model answer:

```
Waits-for graph:
- T1 wants S(B), but T2 holds X(B) → T1 waits for T2 → Edge: T1 → T2
- T2 wants S(C), but T3 holds X(C) → T2 waits for T3 → Edge: T2 → T3
- T3 wants S(A), but T1 holds X(A) → T3 waits for T1 → Edge: T3 → T1

Cycle: T1 → T2 → T3 → T1

YES, this is a deadlock. No transaction can proceed.
The DBMS must abort one transaction (victim) to break the cycle.
```

This is a 3-way deadlock forming a cycle T1→T2→T3→T1. Each transaction holds a lock the next one needs, and needs a lock the previous one holds. The system detects the cycle in the waits-for graph and aborts one transaction.

11 SERIALIZABILITY 5 pts

Given this schedule with three transactions:

```
w1(A), r2(A), w2(B), r3(B), w3(A)
```

List all conflicts, draw the conflict graph edges, and determine if this is conflict serializable. If yes, give an equivalent serial order.

```
List conflicts, edges, and whether it's serializable...
```

No answer provided.

Model answer:

```
Operations in order: w1(A), r2(A), w2(B), r3(B), w3(A)

Conflicts (different txns, same item, at least one write):
On A:
1. w1(A) → r2(A): T1→T2 (WR)
2. w1(A) → w3(A): T1→T3 (WW)
3. r2(A) → w3(A): T2→T3 (RW)

On B:
4. w2(B) → r3(B): T2→T3 (WR)

Conflict graph edges: T1→T2, T1→T3, T2→T3
All arrows go forward: T1→T2→T3. No cycles.

YES, conflict serializable. Equivalent serial order: T1, T2, T3.
```

All edges point in one direction (T1→T2→T3), forming a DAG. The topological sort gives the serial order T1, T2, T3. No cycle means the interleaved schedule is safe.

12 BLOOM FILTERS 5 pts

You have a Bloom filter with **m = 10 bits** and **k = 2 hash functions**. After inserting elements X and Y, the bit array is: `[0, 1, 0, 1, 0, 0, 1, 0, 1, 0]` . You query element Z and get $h_1(Z) = 1$, $h_2(Z) = 6$. What does the Bloom filter report?

- ○ a) Z is definitely in the set
- ◉ b) Z is possibly in the set (false positive possible)
- ○ c) Z is definitely NOT in the set
- ○ d) Cannot determine without knowing X and Y

Correct! Check positions $h_1(Z)=1$ and $h_2(Z)=6$ (0-indexed). Bit array: [0,1,0,1,0,0,1,0,1,0]. Position 1 = 1 and position 6 = 1 — both are set. The Bloom filter reports "possibly in set." However, Z was never inserted — this is a false positive. Bits 1 and 6 were set by X and Y, not Z. This is exactly how false positives work in Bloom filters.

13 BLOOM FILTERS 5 pts

Why can't you delete elements from a standard Bloom filter by setting bits back to 0?

- ○ a) The hash functions are not reversible
- ○ b) Multiple elements may share the same bit positions; clearing a bit could create false negatives for other elements
- ◉ c) Bloom filters are read-only after construction
- ○ d) The bit array would become all zeros

Incorrect (you picked: C, correct: B). Multiple elements can hash to the same bit position. If element X sets bit 3 and element Y also sets bit 3, clearing bit 3 when deleting X would make the Bloom filter report "definitely not present" for Y — a false negative. Bloom filters guarantee zero false negatives, so deletion would break this fundamental property.

14 INDEXING 5 pts

A table is physically sorted by `id` (primary key). You create a secondary index on `salary` . Must this secondary index be dense or sparse, and why?

- (●) a) Sparse — secondary indexes are always sparse to save space
- ( ) b) Dense — the data is not sorted by salary, so you can't rely on sequential scan from a nearby entry
- ( ) c) Either works — the database optimizer chooses
- ( ) d) Dense — secondary indexes always have more entries than primary indexes

Incorrect (you picked: A, correct: B). A sparse index works by finding the nearest smaller entry and scanning sequentially from there. This only works if the data is physically sorted by the search key. Since the table is sorted by id (not salary), consecutive salary values are scattered across different disk blocks. You need a dense index (one entry per value) so every lookup has a direct pointer. Secondary indexes MUST be dense.

15 CACHING 5 pts

Cache size = 3. Sequence: `A, B, C, A, B, D, A, B, C, D`. Complete the FIFO trace. How many misses occur?

Show the cache state after each request and count misses...

No answer provided.

Model answer:

```
FIFO trace (cache size 3):
A: Miss  [A]
B: Miss  [A, B]
C: Miss  [A, B, C]       ← cache full
A: Hit   [A, B, C]       ← A in cache, no change (FIFO doesn't move on hit)
B: Hit   [A, B, C]       ← B in cache, no change
D: Miss  [B, C, D]       ← evict A (first in), insert D
A: Miss  [C, D, A]       ← evict B (first in), insert A
B: Miss  [D, A, B]       ← evict C (first in), insert B
C: Miss  [A, B, C]       ← evict D (first in), insert C
D: Miss  [B, C, D]       ← evict A (first in), insert D

Result: 7 misses, 3 hits out of 10 requests (30% hit rate)
```

FIFO evicts the oldest inserted item regardless of access pattern. Even though A and B were recently accessed (hits at steps 4-5), FIFO still evicts them when they're oldest in the queue. This is why LRU outperforms FIFO — LRU would refresh their position on a hit.

16 CACHING 5 pts

Cache size = 3. Cache currently holds `{A, B, C}`. Next request is D. Remaining future sequence after D is: `A, C, B, A`. Under OPT (Belady's), which item is evicted?

- ( ) a) A — it was accessed least recently
- (●) b) B — its next access is furthest in the future
- ( ) c) C — it has the lowest frequency count
- ( ) d) A — it won't be accessed again

Correct! OPT looks at when each item is next accessed in the future sequence (A, C, B, A). A is next accessed at position 1 (soon). C is next at position 2. B is next at position 3 (furthest away). OPT evicts the item whose next use is furthest in the future — that's B. Note: OPT doesn't care about recency (LRU) or frequency (LFU), only the future.

17 NETWORKING 5 pts

A video call application uses UDP instead of TCP. Which of the following best explains why?

- ( ) a) UDP encrypts data while TCP does not
- (●) b) UDP is connectionless and doesn't retransmit lost packets, avoiding latency spikes from retransmission delays
- ( ) c) UDP uses smaller port numbers so there's less overhead
- ( ) d) UDP guarantees packet ordering which is critical for video

Correct! For real-time video, a slightly dropped frame is better than waiting for a retransmission. TCP guarantees delivery by retransmitting lost packets, which introduces variable delays (jitter). UDP just sends and moves on — if a packet is lost, the video skips a frame rather than freezing while waiting for retransmission. For real-time applications, low and predictable latency matters more than perfect delivery.

18 NETWORKING 5 pts

When your laptop connects to a new Wi-Fi network, it doesn't have an IP address yet. Which protocol assigns it one, and at which layer does this operate?

- ○ a) DNS at the Application layer
- ◉ b) ARP at the Data Link layer
- ○ c) DHCP at the Data Link layer
- ○ d) BGP at the Network layer

Incorrect (you picked: B, correct: C). DHCP (Dynamic Host Configuration Protocol) assigns your laptop an IP address through a 4-step handshake: DISCOVER → OFFER → REQUEST → ACK. It operates at Layer 2 (Data Link) because your laptop doesn't have an IP address yet — it must use broadcast MAC frames to communicate. ARP maps IP→MAC (doesn't assign IPs). DNS maps hostnames→IPs. BGP is for inter-network routing.

19 DISTRIBUTED SYSTEMS 5 pts

In Two-Phase Commit, the coordinator sends PREPARE and all participants vote YES. The coordinator writes COMMIT to its log and starts sending COMMIT messages. It crashes after sending COMMIT to participant 1 but before sending it to participant 2. What happens when the coordinator recovers?

- ◉ a) The transaction is aborted since not all participants received COMMIT
- ○ b) The coordinator re-reads its log, sees COMMIT, and re-sends COMMIT to all participants
- ○ c) Participant 2 independently decides to abort after a timeout
- ○ d) The coordinator starts a new PREPARE phase from scratch

Incorrect (you picked: A, correct: B). The coordinator wrote COMMIT to its WAL before crashing. On recovery, it reads the log, sees the COMMIT decision, and re-sends COMMIT to all participants. This is exactly why WAL is used — the decision survives the crash. Participant 2 may have been blocked waiting, but once the coordinator recovers, it completes the protocol. The transaction is NOT aborted because the decision was already made and logged.

20 PARTITIONING 5 pts

You have 4 nodes using hash partitioning with `node = hash(key) mod 4`. You add a 5th node, changing the formula to `mod 5`. Approximately what fraction of keys must be moved?

- ○ a) About 1/5 (20%)
- ○ b) About 4/5 (80%)
- ○ c) All keys (100%)
- ◉ d) None — mod 4 keys still map correctly under mod 5

Incorrect (you picked: D, correct: B). With mod 4 → mod 5, a key stays on the same node only if hash(key) mod 4 = hash(key) mod 5, which happens for about 1/5 of keys (those where mod 4 and mod 5 agree). The other ~4/5 (80%) get rehashed to a different node. This is the problem consistent hashing solves — with a hash ring, adding a 5th node only moves ~1/5 of keys (the ones on the ring segment the new node takes over).

Grade Exam Reset

45 / 100

45% — Keep studying — go through all explanations below.

CSEE 4121 Practice Midterm 2 — Columbia University — Spring 2026